

SQL Fundamentals

Chapter 05 - Sub-Chapter 01

SQL Databases Series

What is SQL?	Relational model, tables, keys, constraints
DDL	CREATE TABLE, ALTER TABLE, data types
DML	INSERT, UPDATE, DELETE safely
SELECT	Filtering, JOINS, aggregation
Subqueries and CTEs	Modular, readable SQL
Normalization	1NF, 2NF, 3NF — eliminating redundancy

1 What is SQL?

The language of relational databases — 50 years strong

SQL (Structured Query Language) was invented at IBM in 1974 and standardized in 1986. Today it powers every major relational database — PostgreSQL, MySQL, Oracle, SQL Server, SQLite — and is the most widely used data language in the world.

The relational model, proposed by Edgar Codd in 1970, stores data in **tables** (relations). Each table has **columns** (attributes) with fixed types, and **rows** (tuples) of data. Tables link to each other through **foreign keys** — creating a web of related data you can query with a single SQL statement.

Relational Model — Tables, Rows, Columns, Keys

users				orders			
user_id PK	name	email	created_at	order_id PK	user_id FK	total	status
1	Alice	alice@ex.com	2024-01-10	101	1	\$89.99	CONFIRMED
2	Bob	bob@ex.com	2024-01-11	102	1	\$45.00	SHIPPED
3	Carol	carol@ex.com	2024-01-12	103	2	\$120.50	PENDING

Primary Key (PK) uniquely identifies a row · Foreign Key (FK) links tables · Referential Integrity enforced

The four categories of SQL commands:

- **DDL (Data Definition Language):** CREATE TABLE, ALTER TABLE, DROP TABLE — define the schema.
- **DML (Data Manipulation Language):** INSERT, UPDATE, DELETE — change the data.
- **DQL (Data Query Language):** SELECT — retrieve data.
- **DCL (Data Control Language):** GRANT, REVOKE — control access.

2 DDL — Defining Your Schema

CREATE, ALTER, DROP with real examples

Schema design is permanent — changing a table structure after launch is expensive. Get the data types and constraints right from the start.

Creating tables for an e-commerce system:

```
-- Users table with proper types and constraints
CREATE TABLE users (
  user_id    BIGSERIAL PRIMARY KEY,          -- auto-incrementing integer
  email      VARCHAR(255) NOT NULL UNIQUE,   -- must exist, must be unique
  name       VARCHAR(100) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  status     VARCHAR(20)  NOT NULL DEFAULT 'active'
              CHECK (status IN ('active','suspended','deleted'))
);

-- Orders table with foreign key back to users
CREATE TABLE orders (
  order_id   BIGSERIAL PRIMARY KEY,
  user_id    BIGINT NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
  status     VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  total      NUMERIC(10, 2) NOT NULL CHECK (total >= 0),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  shipped_at  TIMESTAMPTZ          -- nullable: set when order ships
);

-- Order items (many-to-one with orders, many-to-one with products)
CREATE TABLE order_items (
  item_id    BIGSERIAL PRIMARY KEY,
  order_id   BIGINT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
  sku        VARCHAR(50) NOT NULL,
  qty        INTEGER NOT NULL CHECK (qty > 0),
  unit_price NUMERIC(10, 2) NOT NULL CHECK (unit_price >= 0)
);
```

ALTER TABLE — adding/changing columns after creation:

```
-- Add a new column (existing rows get NULL unless DEFAULT provided)
ALTER TABLE users ADD COLUMN phone VARCHAR(20);

-- Add a NOT NULL column with a default (safe on existing data)
ALTER TABLE orders ADD COLUMN currency VARCHAR(3) NOT NULL DEFAULT 'USD';

-- Rename a column
ALTER TABLE users RENAME COLUMN name TO full_name;

-- Change a column type (careful! may fail if data is incompatible)
ALTER TABLE orders ALTER COLUMN total TYPE NUMERIC(12, 2);

-- Add a constraint after the fact
ALTER TABLE users ADD CONSTRAINT chk_email_format
  CHECK (email LIKE '%@%.%');

-- Create an index (covered in detail in Sub-Chapter 02)
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_status  ON orders(status);
```

3 DML — Inserting, Updating, Deleting

Writing data safely and efficiently

INSERT — adding data:

```
-- Insert a single row
INSERT INTO users (email, name, status)
VALUES ('alice@example.com', 'Alice Johnson', 'active');

-- Insert multiple rows in one statement (much faster than individual inserts)
INSERT INTO order_items (order_id, sku, qty, unit_price)
VALUES
    (101, 'BOOT-42', 1, 89.99),
    (101, 'SOCK-L', 2, 7.50),
    (101, 'LACE-BK', 1, 3.99);

-- INSERT ... ON CONFLICT (upsert) -- insert or update if exists
INSERT INTO users (email, name)
VALUES ('alice@example.com', 'Alice Smith')
ON CONFLICT (email)
DO UPDATE SET name = EXCLUDED.name,      -- update with new value
              updated_at = NOW();

-- INSERT ... SELECT -- copy data from another table
INSERT INTO order_archive (order_id, user_id, total, archived_at)
SELECT order_id, user_id, total, NOW()
FROM orders
WHERE created_at < NOW() - INTERVAL '2 years';
```

UPDATE — modifying existing rows:

```
-- Update specific rows (ALWAYS use WHERE -- without it, ALL rows change!)
UPDATE orders
SET status      = 'SHIPPED',
    shipped_at = NOW()
WHERE order_id = 101;

-- Update based on a join -- mark orders as 'VIP' for premium users
UPDATE orders o
SET status = 'VIP_' || o.status
FROM users u
WHERE o.user_id = u.user_id
    AND u.tier = 'premium'
    AND o.status = 'CONFIRMED';

-- Conditional update with CASE
UPDATE products
SET price = CASE
    WHEN category = 'electronics' THEN price * 0.90 -- 10% off
    WHEN category = 'clothing'     THEN price * 0.85 -- 15% off
    ELSE price
END
WHERE sale_active = true;
```

DELETE — removing rows:

```
-- Delete specific rows (ALWAYS use WHERE!)
DELETE FROM order_items WHERE order_id = 101;
DELETE FROM orders WHERE order_id = 101;
```

```
-- Soft delete -- preferred in production (never actually delete)
UPDATE users SET status = 'deleted', deleted_at = NOW()
WHERE user_id = 42;

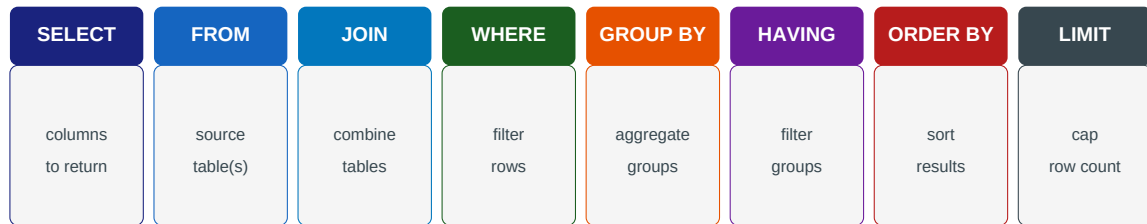
-- Delete with subquery -- remove orphaned order items
DELETE FROM order_items
WHERE order_id NOT IN (SELECT order_id FROM orders);

-- TRUNCATE -- delete ALL rows, much faster than DELETE (no WHERE!)
TRUNCATE TABLE sessions; -- instant, non-recoverable!
TRUNCATE TABLE order_items RESTART IDENTITY CASCADE; -- reset sequences too
```

4 SELECT — Querying Data

FROM basics to subqueries and CTEs

Anatomy of a SELECT Statement



Execution order: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Basic SELECT — filtering, sorting, limiting:

```
-- Get all confirmed orders for a user, newest first, limit 10
SELECT order_id, total, status, created_at
FROM orders
WHERE user_id = 42
  AND status = 'CONFIRMED'
  AND created_at > NOW() - INTERVAL '30 days'
ORDER BY created_at DESC
LIMIT 10;

-- Calculated columns and aliases
SELECT
  order_id,
  total,
  total * 0.18 AS tax_amount,
  total * 1.18 AS total_with_tax,
  UPPER(status) AS status_display,
  DATE_TRUNC('day', created_at) AS order_date
FROM orders
WHERE user_id = 42;

-- DISTINCT -- remove duplicate values
SELECT DISTINCT status FROM orders;
SELECT DISTINCT ON (user_id) user_id, order_id, created_at
FROM orders ORDER BY user_id, created_at DESC; -- latest order per user
```

JOINS — combining tables:

```
-- INNER JOIN -- only rows that match in BOTH tables
SELECT u.name, u.email, o.order_id, o.total
FROM users u
INNER JOIN orders o ON u.user_id = o.user_id
WHERE o.status = 'CONFIRMED';

-- LEFT JOIN -- all users, even those with no orders
SELECT u.name, COUNT(o.order_id) AS order_count,
  COALESCE(SUM(o.total), 0) AS lifetime_value
FROM users u
LEFT JOIN orders o ON u.user_id = o.user_id
GROUP BY u.user_id, u.name
ORDER BY lifetime_value DESC;

-- Multi-table join
SELECT u.name, o.order_id, oi.sku, oi.qty, oi.unit_price
```

```
FROM users u
JOIN orders o      ON u.user_id = o.user_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE u.user_id = 42
ORDER BY o.created_at DESC, oi.sku;
```

Aggregation — GROUP BY and HAVING:

```
-- Sales summary: total revenue and order count per status
SELECT
    status,
    COUNT(*)           AS order_count,
    SUM(total)         AS total_revenue,
    AVG(total)         AS avg_order_value,
    MIN(total)         AS min_order,
    MAX(total)         AS max_order
FROM orders
WHERE created_at >= DATE_TRUNC('month', NOW()) -- this month
GROUP BY status
HAVING COUNT(*) > 10 -- only statuses with more than 10 orders
ORDER BY total_revenue DESC;

-- Sales by day for the last 30 days
SELECT
    DATE_TRUNC('day', created_at) AS sale_date,
    COUNT(*) AS orders,
    SUM(total) AS revenue
FROM orders
WHERE created_at > NOW() - INTERVAL '30 days'
GROUP BY sale_date
ORDER BY sale_date;
```

5 Subqueries and CTEs

Writing readable, modular SQL

Complex business questions often require building a result step by step. Subqueries and CTEs (Common Table Expressions) let you name intermediate results and compose them — making SQL readable and maintainable.

Subquery — inline query inside another query:

```
-- Find users who have placed at least one order over $200
SELECT name, email
FROM users
WHERE user_id IN (
    SELECT DISTINCT user_id
    FROM orders
    WHERE total > 200
);

-- Correlated subquery -- references the outer query per row (slower!)
-- Find each user's most recent order
SELECT u.name,
    (SELECT MAX(o.created_at)
     FROM orders o
     WHERE o.user_id = u.user_id) AS last_order_date
FROM users u;

-- Scalar subquery in SELECT -- total orders as a column
SELECT
    order_id, total,
    (SELECT COUNT(*) FROM order_items oi
     WHERE oi.order_id = o.order_id) AS item_count
FROM orders o
WHERE user_id = 42;
```

CTE (WITH clause) — named, reusable intermediate results:

```
-- CTEs make complex queries readable -- executed top to bottom
WITH
-- Step 1: get orders from this month
monthly_orders AS (
    SELECT user_id, order_id, total
    FROM orders
    WHERE created_at >= DATE_TRUNC('month', NOW())
    AND status = 'CONFIRMED'
),
-- Step 2: aggregate per user
user_spending AS (
    SELECT user_id,
        COUNT(*) AS order_count,
        SUM(total) AS monthly_spend
    FROM monthly_orders
    GROUP BY user_id
),
-- Step 3: rank users
ranked_users AS (
    SELECT user_id, order_count, monthly_spend,
        RANK() OVER (ORDER BY monthly_spend DESC) AS rank
    FROM user_spending
)
```



```
-- Step 4: join back to get names, keep top 10
SELECT u.name, u.email, r.order_count, r.monthly_spend, r.rank
FROM ranked_users r
JOIN users u ON r.user_id = u.user_id
WHERE r.rank <= 10
ORDER BY r.rank;
```

6 Normalization

Designing tables that do not lie

Normalization is the process of organizing a database to reduce redundancy and prevent update anomalies. An **update anomaly** occurs when the same fact is stored in multiple places — update one, miss another, and your database contains contradictions.

Normalization — Eliminating Redundancy

Unnormalized (bad)

uid	name	email	oid	total	item
1	Alice	alice@x.com	101	\$89	Book
1	Alice	alice@x.com	102	\$45	Pen
2	Bob	bob@x.com	103	\$20	Ink

Problem: Alice's email repeated 2x → update anomaly!

Normalized (good)

- users(uid PK, name, email)
- orders(oid PK, uid FK, total)
- items(iid PK, oid FK, name)

✓ Each fact stored once — no anomalies

The three main normal forms:

1NF (First Normal Form): Every column holds atomic (single, indivisible) values. No repeating groups. Each row is unique. Violation example: storing 'Alice, Bob' in a single 'passengers' column.

2NF (Second Normal Form): In 1NF, AND every non-key column depends on the *whole* primary key. Only relevant when the primary key is composite (multi-column). Violation example: order_items table with (order_id, product_id) PK but storing product_name — which depends only on product_id, not the full key.

3NF (Third Normal Form): In 2NF, AND no non-key column depends on another non-key column (transitive dependency). Violation example: storing zip_code AND city — city depends on zip_code, not on the PK. Fix: extract zip codes to their own table.

```
-- 3NF Design: extract all transitive dependencies

-- BEFORE (3NF violation): city stored with user, depends on zip
-- users(user_id, name, zip_code, city)    -- city depends on zip, not user_id!

-- AFTER (3NF compliant):
CREATE TABLE zip_codes (
    zip_code VARCHAR(10) PRIMARY KEY,
    city     VARCHAR(100) NOT NULL,
    state    VARCHAR(50)  NOT NULL
);
CREATE TABLE users (
    user_id BIGSERIAL PRIMARY KEY,
    name     VARCHAR(100) NOT NULL,
    zip_code VARCHAR(10) REFERENCES zip_codes(zip_code)
    -- city is derived from zip_code -- not stored here
);
-- Now updating a city affects one row in zip_codes, not every user
```

Key Rules

- Always define a PRIMARY KEY -- it uniquely identifies each row and creates an index automatically.

- Use NOT NULL constraints aggressively -- NULL is a special value that causes subtle bugs.
- Define FOREIGN KEY constraints -- the database enforces referential integrity for you.
- Use CHECK constraints to validate data at the database level (not just application level).
- Normalize to 3NF by default -- denormalize only when query performance demands it.
- Use TIMESTAMPTZ (with timezone) not TIMESTAMP -- always store timestamps in UTC.